



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1
по дисциплине «Анализ Алгоритмов»

Тема Расстояние Левенштейна

Студент Поляков А.И.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Д.М.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Расстояние Левенштейна	5
1.1.1 Рекуррентная формула расстояния Левенштейна	5
1.1.2 Алгоритм Левенштейна с кешированием	6
1.2 Алгоритм Дамерау-Левенштейна	7
1.2.1 Рекурсивная формула расстояния Дамерау-Левенштейна	7
1.2.2 Алгоритм Дамерау-Левенштейна с кешированием	8
2 Конструкторская часть	9
2.1 Требования к программному обеспечению	9
2.2 Разработка алгоритмов	10
2.3 Вывод	12
3 Технологическая часть	13
3.1 Средства разработки	13
3.2 Реализация алгоритмов	13
3.3 Функциональные тесты	15
4 Исследовательская часть	17
4.1 Технические характеристики	17
4.2 Временные характеристики	17
4.3 Емкостные характеристики	19
4.4 Вывод	21
ЗАКЛЮЧЕНИЕ	22

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23
---	-----------

ВВЕДЕНИЕ

Расстояние Левенштейна – минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Оно используется для нечеткого поиска, коррекции опечаток, обработки текста и сравнения последовательностей, например, в биоинформатике и текстовых приложениях.

Целью данной лабораторной работы является проведение исследования метода динамического программирования на примере алгоритмов Левенштейна и Дамерау-Левенштейна.

Задачами данной лабораторной являются:

- 1) Рассмотрение алгоритмов Левенштейна и Дамерау-Левенштейна нахождения расстояния между строками;
- 2) Разработка рекурсивного алгоритма расстояния Левенштейна, а также применение методов динамического программирования для разработки алгоритмов расстояний Левенштейна и Дамерау-Левенштейна с использованием кеширования;
- 3) Получение практических навыков реализации указанных алгоритмов: двух алгоритмов с кешированием и одного из алгоритмов в рекурсивной версии;
- 4) Экспериментальное подтверждение различий во временной эффективности рекурсивной и итерационной реализаций алгоритма определения расстояния Левенштейна с помощью замеров процессорного времени и пиковой памяти при выполнении алгоритмов на варьирующихся длинах строк.

1 Аналитическая часть

1.1 Расстояние Левенштейна

Расстояние Левенштейна – это метрика, используемая для определения минимального количества элементарных операций, необходимых для преобразования одной строки (последовательности символов) в другую. К таким операциям относятся: вставка, удаление и замена символа.

Расстояние Левенштейна широко применяется в задачах, связанных с обработкой текста и последовательностей данных. Например, в нечетком поиске, оно позволяет определить степень схожести строк, что помогает при поиске слов с опечатками. В коррекции орфографии расстояние Левенштейна используется для нахождения слов, которые максимально близки к заданному слову с ошибкой. В области обработки естественного языка (NLP) метрика помогает анализировать и сопоставлять предложения, тексты и другие текстовые структуры, чтобы оценить их схожесть.

Кроме того, расстояние Левенштейна находит применение в биоинформатике для сравнения последовательностей ДНК и белков, а также в системах контроля версий для отслеживания различий между файлами и их изменениями. [6]

1.1.1 Рекуррентная формула расстояния Левенштейна

Пусть существует две последовательности символов S_1 и S_2 длины N и M соответственно. Введем функцию $D(i, j)$, равную расстоянию Левенштейна между подстроками $S_1[1...i]$ и $S_2[1...j]$.

Тогда расстояние Левенштейна для двух строк может быть выражено следующей рекуррентной формулой:

$$D(i, j) = \begin{cases} 0 & i = 0 \text{ и } j = 0 \\ i & i > 0 \text{ и } j = 0 \\ j & i = 0 \text{ и } j > 0 \\ \min \begin{cases} D(i, j - 1) + 1 \\ D(i - 1, j) + 1 \\ D(i - 1, j - 1) + f_e(S_1[i], S_2[j]) \end{cases} & i > 0 \text{ и } j > 0 \end{cases} \quad (1.1)$$

При этом

$$f_e(a1, a2) = \begin{cases} 0 & a1 == a2 \\ 1 & a1 != a2 \end{cases} \quad (1.2)$$

Из функции видно, что она вычисляет минимальное расстояние Левенштейна на текущем этапе (по индексу в первой строке), постепенно приводя первую строку к виду второй с конца. В рекурсивной части: первый случай символизирует вставку символа из второй строки в первую, второй — удаление символа из первой строки, а третий — замену символа первой строки на символ второй. Если символы уже совпадают, замена не требуется и её стоимость равна 0. [5]

1.1.2 Алгоритм Левенштейна с кешированием

Для избежания многократного вычисления $D(i, j)$ для одинаковых аргументов, используют итерационный алгоритм, который сохраняет промежуточные результаты в матрице размером $(N + 1) \times (M + 1)$.

Каждая ячейка матрицы $[i, j]$ хранит значение $D(i, j)$, представляющее расстояние Левенштейна между подстроками $S_1[1...i]$ и $S_2[1...j]$. Нулевая строка и нулевой столбец матрицы заполняются от 0 до соответствующего индекса, так как для преобразования пустой строки в строку длины i требуется i операций вставки.

Затем алгоритм построчно заполняет матрицу по формуле (1.1), используя уже сохранённые значения из матрицы, что исключает необходимость пересчёта значений D .

Для оптимизации памяти на каждой итерации сохраняют только текущую строку и предыдущую, так как остальные строки не участвуют в дальнейшем вычислении. [5]

1.2 Алгоритм Дамерау-Левенштейна

Расстояние Дамерау-Левенштейна – метрика, которая измеряет расстояние между двумя последовательностями символов, определяя минимальное количество операций (вставка, замена, удаление и транспозиция), необходимых для преобразования одной последовательности в другую. Транспозиция подразумевает перестановку двух соседних символов. В отличие от расстояния Левенштейна, эта метрика включает операцию транспозиции, что может значительно уменьшить итоговое расстояние между последовательностями.

1.2.1 Рекурсивная формула расстояния

Дамерау-Левенштейна

Введем также как и для расстояния Левенштейна функцию $D(i, j)$, при этом с учётом транспозиции формула будет выглядеть так:

$$D(i, j) = \begin{cases} 0 & i = 0, j = 0 \\ i & i > 0, j = 0 \\ j & i = 0, j > 0 \\ \min \begin{cases} D(i, j - 1) + 1 \\ D(i - 1, j) + 1 \\ D(i - 1, j - 1) + f_e(S_1[i], S_2[j]) \\ D(i - 2, j - 2) + 1 \end{cases} & \begin{matrix} i > 1, j > 1, \\ S_1[i] = S_2[j - 1], \\ S_1[i - 1] = S_2[j] \end{matrix} \\ \min \begin{cases} D(i, j - 1) + 1 \\ D(i - 1, j) + 1 \\ D(i - 1, j - 1) + f_e(S_1[i], S_2[j]) \end{cases} & \text{Иначе} \end{cases} \quad (1.3)$$

Фактическое отличие от алгоритма расстояния Левенштейна в том, что

при $j > 1$ и $i > 1$, а также при условии что соседние символы в строках равны «накрест», может быть проведена операция транспозиции, которая может сократить расстояние.

1.2.2 Алгоритм Дамерау-Левенштейна с кешированием

Существует итерационный алгоритм с кешированием, сохраняющий промежуточные значения в матрице либо только текущую и последние две строки. Алгоритм схож с алгоритмом для вычисления расстояния Левенштейна, за исключением того, что при совпадении двух соседних символов строк «накрест» (если $S_1[i] = S_2[j - 1]$ и $S_1[i - 1] = S_2[j]$) и при $i > 1$ и $j > 1$, необходимо учитывать вариант с транспозицией при расчёте текущей ячейки матрицы. Это позволяет учесть операцию перестановки символов, что может уменьшить итоговое расстояние. [5]

Вывод

В результате аналитического раздела были рассмотрены расстояния Левенштейна и Дамерау-Левенштейна, а также их рекурсивные формулы, итерационные алгоритмы с кешированием матрицами и отдельными строками матриц.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемой программе предъявлен ряд требований:

Входные данные: Две строки

Выходные данные: Целое число

- Заглавные и строчные символы считаются разными;
- Возможность обработки строк как на кириллице так и на латинице;
- Должна быть возможность замера процессорного времени программы;
- Должна быть возможность вывода графиков и таблиц замеров процессорного времени и памяти.

2.2 Разработка алгоритмов

На рисунке 2.1 представлена схема рекурсивного алгоритма нахождения расстояния Левенштейна.

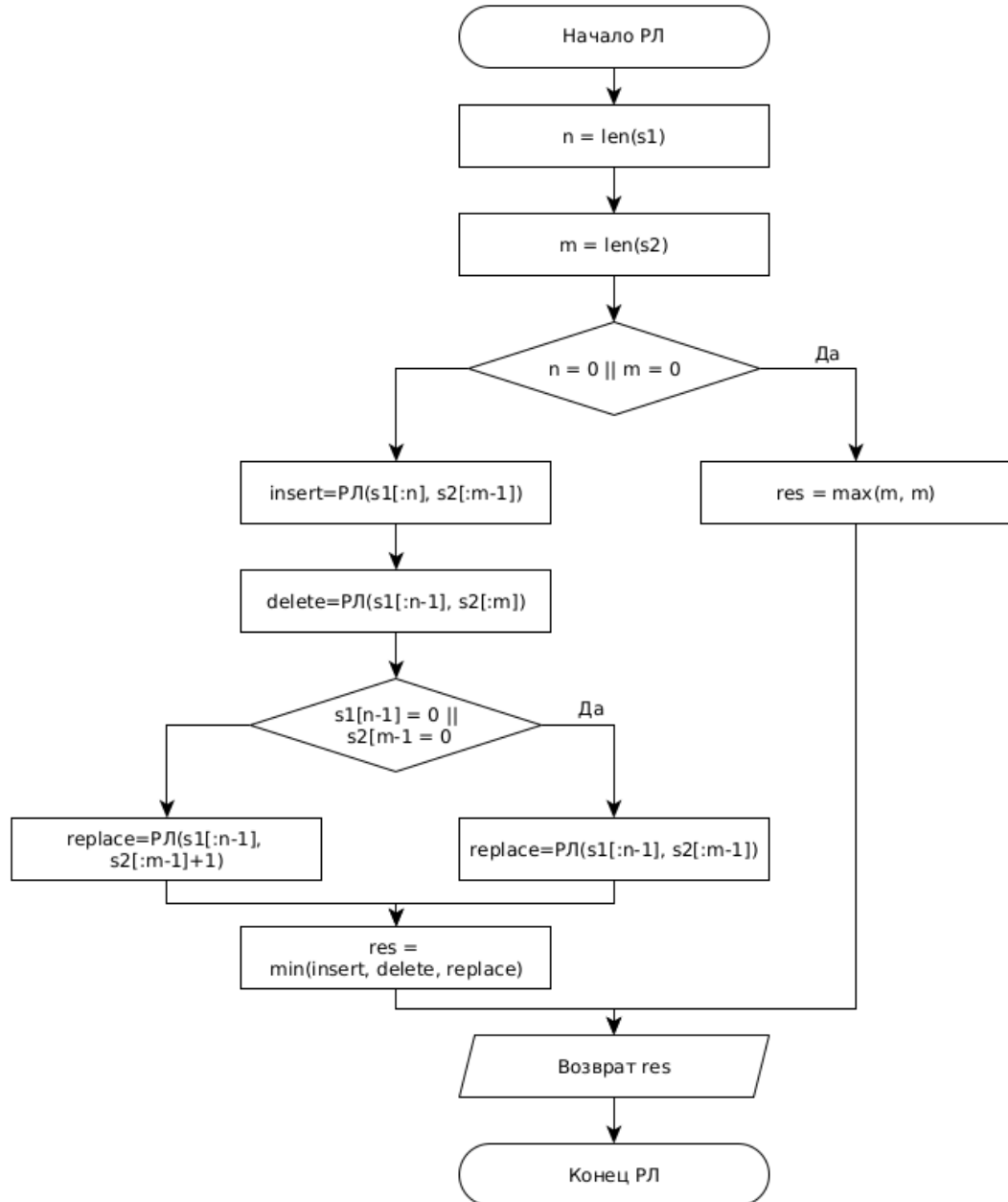


Рисунок 2.1 — Схема рекурсивного алгоритма нахождения расстояния Левенштейна

На рисунке 2.2 представлена схема алгоритма нахождения расстояния Левенштейна с кешем.

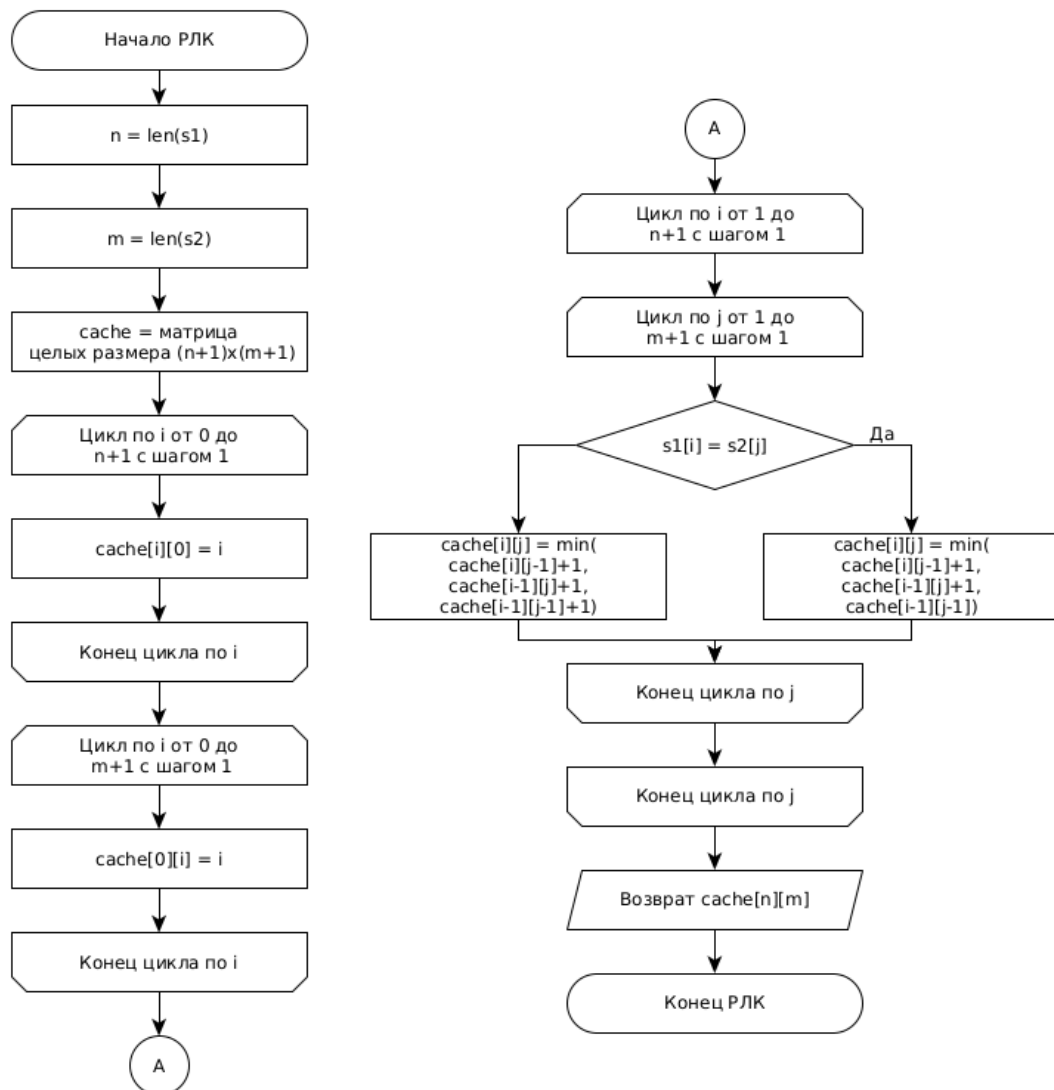


Рисунок 2.2 — Схема алгоритма нахождения расстояния Левенштейна с кешем

На рисунке 2.3 представлена схема алгоритма нахождения расстояния Дameraу-Левенштейна с кешем.

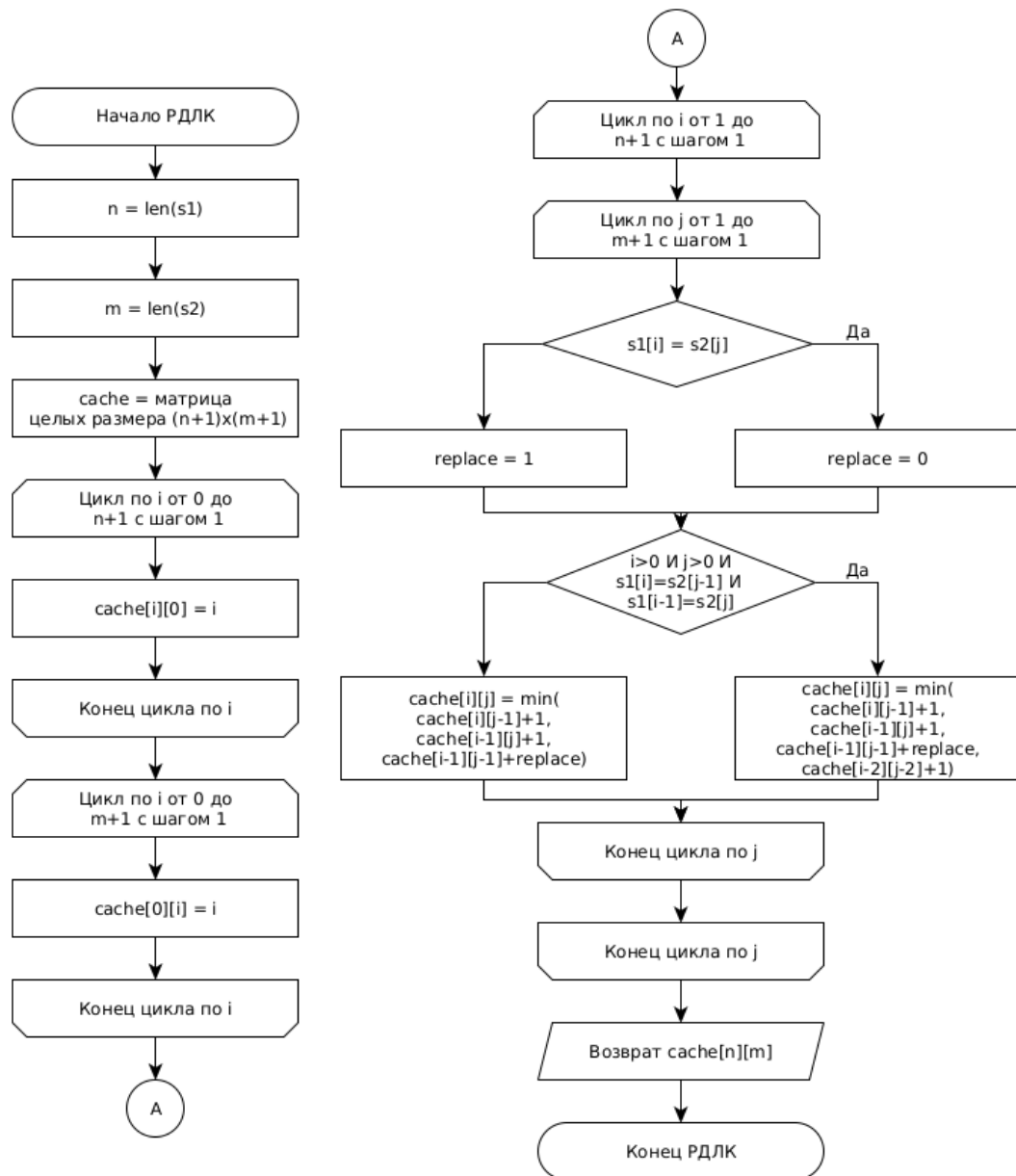


Рисунок 2.3 — Схема алгоритма нахождения расстояния Дameraу-Левенштейна с кешем

2.3 Вывод

В результате конструкторской части были определены требования к ПО, а также сделаны схемы алгоритмов рекурсивного поиска расстояния Левенштейна, итерационного поиска с кешем и итерационного алгоритма поиска расстояния Дameraу-Левенштейна с кешем.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран python3 [1], так как в его стандартной библиотеке присутствуют функции замера процессорного времени и использованной памяти, которые требуются в условиях, а также данный язык обладающий множеством инструментов для визуализации и работы с данными и таблицами.

Для построения графиков использовалась библиотека matplotlib [3].

Для замера времени использовалась функция `process_time_ns` из стандартного модуля `time` [2].

Для замеров памяти использовалась функция `get_traced_memory` стандартного модуля `tracemalloc` [4], которая получает текущую и пиковую выделенную память относительно начала замера.

3.2 Реализация алгоритмов

В листинге 3.1 представлена реализация рекурсивного алгоритма нахождения расстояния Левенштейна.

Листинг 3.1 — Рекурсивный алгоритм нахождения расстояния Левенштейна

```
def reqLev(s1, s2):
    if len(s2) == 0:
        return len(s1)
    elif len(s1) == 0:
        return len(s2)
    elif s1[0] == s2[0]:
        return reqLev(s1[1:], s2[1:])
    else:
        return 1 + min(reqLev(s1[1:], s2), reqLev(s1[1:], s2[1:]),
                        reqLev(s1, s2[1:]))
```

В листинге 3.2 представлена реализация алгоритма нахождения расстояния Левенштейна с кешем.

Листинг 3.2 — Алгоритм нахождения расстояния Левенштейна с кешем

```
def memLev(s1, s2):
    n, m = len(s1), len(s2)
    if n > m:
        s1, s2 = s2, s1
        n, m = m, n

    curr = range(n + 1)
    for i in range(1, m + 1):
        prev, curr = curr, [i] + [0] * n
        for j in range(1, n + 1):
            add, delete, change = prev[j] + 1, curr[j - 1] + 1, prev[j - 1]
            if s1[j - 1] != s2[i - 1]:
                change += 1
            curr[j] = min(add, delete, change)

    return curr[n]
```

В листинге 3.3 представлена реализация алгоритма нахождения расстояния Дамерау-Левенштейна с кешем.

Листинг 3.3 — Итерационный алгоритм нахождения расстояния Дамерау-Левенштейна с кешем

```
def dLev(s1, s2):
    d = {}

    for i in range(-1, len(s1)+1):
        d[(i, -1)] = i+1
    for j in range(-1, len(s2)+1):
        d[(-1, j)] = j+1

    for i in range(len(s1)):
        for j in range(len(s2)):
            if s1[i] == s2[j]:
                cost = 0
            else:
                cost = 1
            d[(i, j)] = min(
                d[(i-1, j)] + 1,
                d[(i, j-1)] + 1,
                d[(i-1, j-1)] + cost,
            )
            if i and j and s1[i] == s2[j-1] and s1[i-1] == s2[j]:
                d[(i, j)] = min(d[(i, j)], d[(i-2, j-2)] + 1)

    return d[len(s1)-1, len(s2)-1]
```

3.3 Функциональные тесты

В таблице 3.1 приведены тесты для алгоритмов расстояние Левенштейна и Дамерау-Левенштейна.

Таблица 3.1 — Функциональные тесты

Входные данные		Расстояния и алгоритм				
Строка 1	Строка 2	Левенштейн			Дамерау-Левенштейн	
		Рекурсивный	С кешем	Ожидаемое	С кешем	Ожидаемое
андрей	дмитрий	4	4	4	4	4
дмитрий	юрий	2	2	2	2	2
река	мука	2	2	2	2	2
1234	2134	3	3	3	2	2

Все тесты пройдены успешно

Вывод

В ходе работы были разработаны рекурсивный алгоритм поиска расстояния Левенштейна и алгоритмы поиска расстояний Левенштейна и Дамерау-Левенштейна с кешем, а также проведено их тестирование.

4 Исследовательская часть

4.1 Технические характеристики

Технические характеристики устройства, на котором проводились замеры:

- AMD Ryzen 7 5800U (16) @ 4.51 ГГц;
- Оперативная память: 16 Гб;
- Ядро ОС Linux 6.10.10-arch1-1.

При проведении замеров ноутбук был включен в сеть и были запущены только системные приложения и реализованные программы.

4.2 Временные характеристики

В таблице 4.1 приведены временные характеристики, полученные в результате замеров времени алгоритмов. Для каждой длины замер проводился 100 раз, при этом для каждой итерации создавались 2 случайные строки заданной длины и использовались как аргументы к алгоритмам. Итоговое время взято как среднее арифметическое всех полученных замеров.

Таблица 4.1 — Временные характеристики

Длина строки, символы	Рекурсивный алгоритм, нс	Алгоритм с кешем, нс
2	2 915.99	2 227.65
3	11 837.55	3 451.7
4	55 621.85	4 863.58
5	257 908.21	6 819.9
6	1 375 881.37	9 031.91
7	6 948 788.65	11 651.07
8	35 773 610.35	14 515.6

Полученные замеры также можно увидеть на рисунке 4.1:

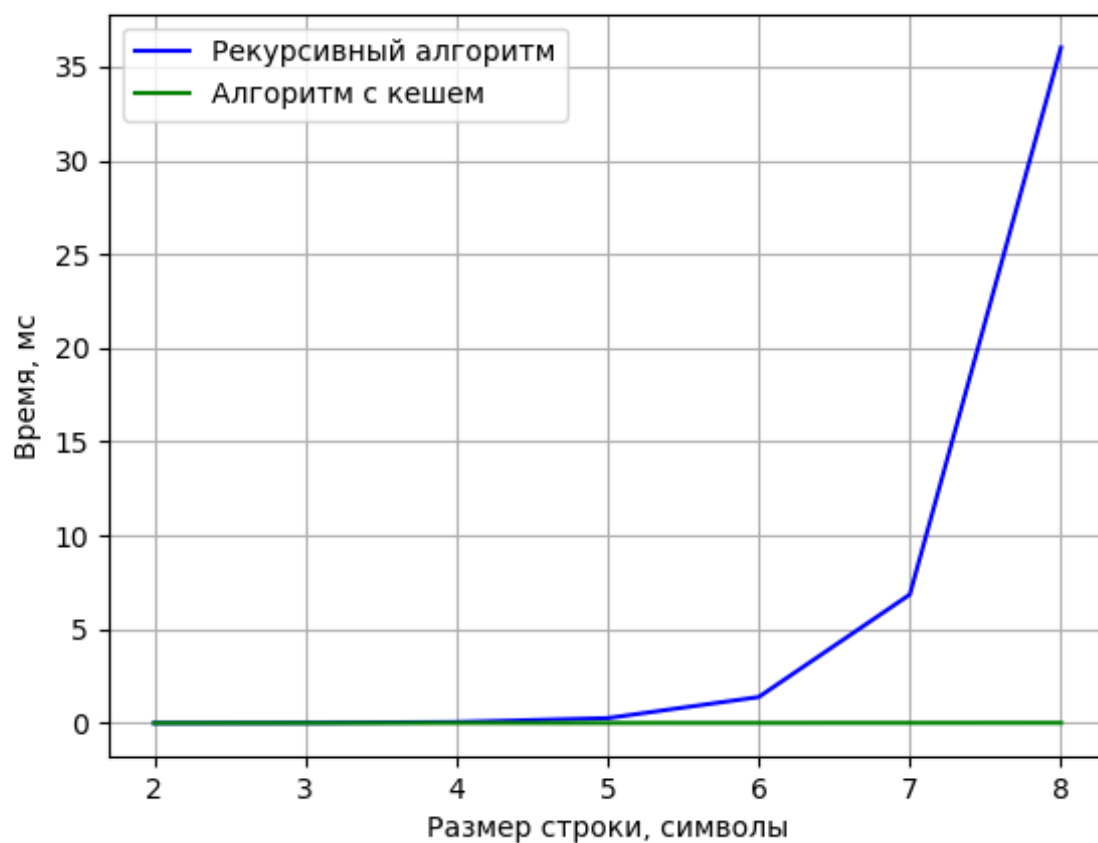


Рисунок 4.1 — График зависимости времени работы алгоритмов от длины строк

Как видно из таблицы 4.1 и рисунка 4.1, рекурсивный алгоритм существенно уступает итерационному по временным характеристикам.

4.3 Емкостные характеристики

В таблице 4.2 приведены временные характеристики, полученные в результате замеров памяти алгоритмов с помощью модуля `tracemalloc`. Так как для рекурсивной реализации размер выделяемой памяти существенно зависит от глубины рекурсии, то есть от входных строк, то для каждой длины строки для рекурсивного алгоритма алгоритм был проведен 8 раз на случайных строках. Для итерационного алгоритма выделяемая память не зависит от входных строк (при одинаковой длине), так как размер матрицы кеша зависит только от размеров строк. За характеристику памяти взята пиковая память, то есть максимальный объем памяти выделенный функции во время её выполнения.

Таблица 4.2 — Емкостные характеристики

Длина строки, символы	Рекурсивный алгоритм, байты	Алгоритм с кешем, байты
2	48	232
3	247	288
4	288	392
5	312	480
6	404	584
7	498	704
8	674	904

Также зависимость из таблицы отражена на рисунке 4.2.

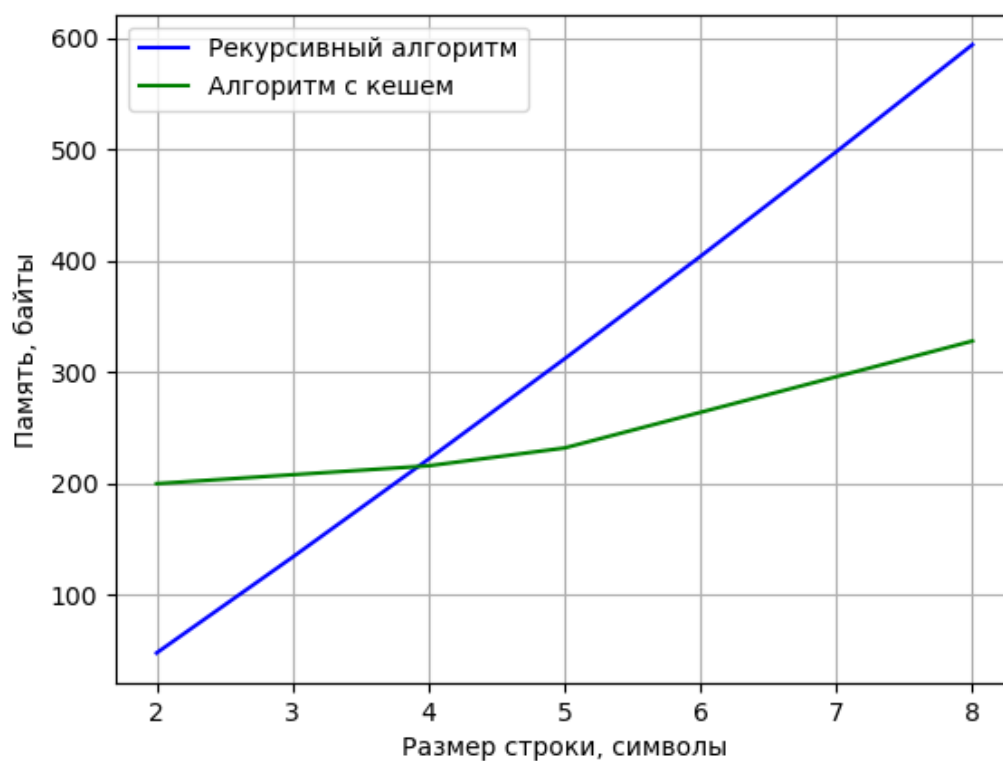


Рисунок 4.2 — График зависимости требуемой памяти для работы алгоритмов от длины входных строк

Как видно из рисунка 4.2 и таблицы 4.2, при небольших размерах строки рекурсивная реализация выигрывает итерационную, однако начиная с длин строк 4, вариант с кешем становится эффективнее, так как реализация рекурсии требует больший объем памяти под входные строки и вызовы.

4.4 Вывод

В данном разделе было проведено исследование временных и ресурсных характеристик алгоритмов поиска расстояния Левенштейна: рекурсивного и итерационного с кешированием.

Из исследования можно сделать вывод, что по времени выполнения рекурсивный алгоритм значительно уступает итерационному. Однако, по использованию ресурсов, при длине входных строк ≤ 3 рекурсивный подход оказывается более эффективным. С увеличением длины строк до ≥ 4 итерационный метод становится предпочтительнее, поскольку рекурсивный алгоритм вызывает множество функций, что приводит к увеличению вычислительных затрат и нагрузки на стек вызовов.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы было проведено исследование метода динамического программирования на примере алгоритмов Левенштейна и Дамерау-Левенштейна.

Выполненные задачи:

- 1) Были рассмотрены алгоритмы Левенштейна и Дамерау-Левенштейна нахождения расстояния между строками;
- 2) Был разработан рекурсивный алгоритм нахождения расстояния Левенштейна, а также алгоритмы нахождения расстояний Левенштейна и Дамерау-Левенштейна с использованием кеширования;
- 3) Реализованы разработанные алгоритмы;
- 4) Произведены замеры процессорного времени и пиковой памяти при выполнении алгоритмов на варьирующихся длинах строк.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python / [Электронный ресурс] // Python : [сайт]. — URL: <https://www.python.org/> (дата обращения: 24.09.2024).
2. time — Time access and conversions / [Электронный ресурс] // Python 3.12.6 documentation : [сайт]. — URL: https://docs.python.org/3/library/time.html#time.process_time_ns (дата обращения: 24.09.2024).
3. Matplotlib, a library for creating visualizations in Python / [Электронный ресурс] // Matplotlib : [сайт]. — URL: <https://matplotlib.org/> (дата обращения: 25.09.2024).
4. tracemalloc — Trace memory allocations / [Электронный ресурс] // Python 3.12.6 documentation : [сайт]. — URL: https://docs.python.org/3/library/tracemalloc.html#tracemalloc.get_traced_memory (дата обращения: 20.09.2024).
5. Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. Алгоритмы: построение и анализ — 3-е изд. — М.: «Вильямс», 2013. — с. 440. — ISBN 978-5-8459-1794-2
6. В. И. Левенштейн (1965). Двоичные коды с исправлением выпадений, вставок и замещений символов. Доклады Академии Наук СССР. 163 (4): 845–848.